Linux 指令练习册 · 进阶篇

系列顺序： 《Linux 入门》→《基础篇》→ 本册。本册场景作业约 10~15 小时（每场景约 3h；场景 2 按需选做，全做约 15h）；完整口径（含复盘与记录）见关卡地图"必做 27.7 小时"。建议先做基础篇。

本册关卡地图（先读这一页，1 分钟）

项目	内容
标签	**课程 + 字典** —— 课程（场景工单）
本册关卡	1-37 ~ 1-45、2-01 ~ 2-12
用时	必做 27.7 小时（按《关卡数据》复算：1-37~1-44 + 2-01~2-12）
学完解锁	拿到一张工单能独立走「现象 → 线索 → 命令 → 验证」，并把 5 件套脚本做出来

场景 2（磁盘满，☆选做）留到以后再做；参考答案约 24,000 字，做完之前不许翻。

关卡编号、时长、完成判定都写在正文每章/每节的「本关三要素」里；全书进度在《17-关卡总表》。

开始之前 · 先弄懂这是什么（2 分钟）

① **是什么**：这是《基础篇》的进阶练习册 —— 把"单个命令"升级成**组合技能**：5 个模拟真实工单的场景（服务起不来 / 磁盘满☆选做 / 服务连不上 / 进程与 GPU 排查 / 脚本收尾），每题**有背景、有故障、只给目标、不给过程**。生活类比：像**医生接急诊**——症状先摆在你面前，怎么查、怎么治要自己想，不是照着菜谱做菜。

② **干什么用**：真实工作里没人给你标准答案，只有现象。这一册练的就是「**现象 → 线索 → 命令 → 验证**」这条排查套路，以及把过程写成记录（故障复盘、讲述稿的底稿都从这儿来）。

③ **为什么要学（+ 实战里怎么用）**：AI 运维的日常就是排障 —— 推理服务挂了、磁盘被模型文件打满、GPU 被别的进程占着，全是这一册的场景；实战里最看重的"讲一件你真正动手做过的事"，答案也在这里。

④ **长什么样 / 怎么用**：最小一条链：现象"服务起不来" → 线索（先看日志最后 20 行）→ 命令（`tail -n 20`、`systemctl status`）→ 验证（`curl` 通了）。规矩照书里：每题**先自己想 3 分钟**、**真敲一遍**、卡住 20 分钟再查。位置：基础篇（02 册）之后、07 Docker 之前。

前言：工单式实战怎么玩

定位：基础篇练"单个技能"，本册练"组合技能"——每道题模拟真实运维工单：**有背景、有故障、只给目标、不给过程**。你扮演值班运维，从一堆现象里自己定位问题、修复、写记录。

答题要求（统一约定）：① **先自己想**——每题至少想 3 分钟再看答案；② **动手敲**——看懂 ≠ 会做，必须真敲一遍、真跑一遍；③ **对答案时看"为什么"**——四步式答案里"为什么这么写"比代码更重要；④ **卡住 20 分钟就查**——先查《Linux 入门》教科书，再查手册，别死磕。

两种用法：① 学完基础篇后集中做；② 边学边做，按需选用。场景 1 和场景 5 与后续的日志排查、Shell 脚本任务衔接最紧密，建议优先完成；场景 2 按需选做。

学习日志：每天把敲过的命令、报错与心得记进学习日志。 说明： = 须电脑环境（Ubuntu）完成；平板 Termux 环境下不确定（不保证可做）。

> ⚠ **安全红线（本册最重要的一节）**：进阶篇所有故障都用**沙盒式故障注入**复刻——故障只发生在练习目录、镜像文件、练习进程里，**不会伤到真实系统**。规则：① 场景 2 结束后必须执行 `sudo umount /mnt/lab`（卸载实验分区）；② 实验目录只用 ~/lab 和 /tmp；③ 每场景做完都要"恢复现场"（杀掉练习进程、删除实验文件）。

实验闭环：所有场景都按“记录原状态 → 核对目标 → 操作 → 对照验证 → 恢复现场”完成。记录目录、文件、PID、端口、挂载和服务状态；异常时保留原始证据，不用宽泛清理命令掩盖失败。

零基础前置约束：本册只用《基础篇》练过的命令；出现新命令（如 `nc`、`fuser`、`nvidia-smi`）时，题目或搭建手册中会先介绍它是什么再使用。答案/附录里首次出现的新命令或选项（如 grep -e、stat、systemctl is-active）同样自带行内解释；其中 `-e` 表示“每个 -e 后跟一个匹配模式”，遇到不认识的先读注释。

本册路线（5 场景）

```
场景1 本地推理服务起不来（日志排查）        约 3h   ★ 必做
场景2 磁盘满应急（模型文件占满磁盘）        约 3h   ☆ 选做
场景3 服务连不上（网络链路排查）            约 3h   ★ 必做
场景4 服务与进程运维（含 GPU 进程定位）     约 3h   ★ 必做
场景5 脚本收尾（衔接 5 个 Shell 脚本）      约 3h   ★ 必做
参考答案（书末集中，先做再看）                     做完题对答案
附录 A · 练习用脚本（labservice.sh）               复刻现场用
附录 B · 本册命令速查                      查询用
附录 C · 术语表补充                        查询用
附录 D · 其余 3 个脚本                     查询用
```

进阶篇 • 场景与题目（只给目标）

场景 1 | 本地推理服务起不来（日志排查）

本关三要素（编号 | 时长 | 完成判定）

1-37 • 场景 1（上）：复现"推理服务起不来" ★ | 休息日 50 分 | 工作日 15 分

□ 在沙盒里把故障复现出来，并写下现象

回滚方式：沙盒故障注入，只动练习目录

1-38 • 场景 1（中）：从日志读到根因 ★ | 休息日 50 分 | 工作日 15 分

□ 说出根因，并能指出日志里哪一行证明了它

1-39 • 场景 1（下）：修复 + 一页处理记录 ★ | 休息日 50 分 | 工作日 15 分

□ 服务起来；记录按"现象→线索→命令→验证"写完

回滚方式：修复后恢复现场 | 实战价值：这些都是复盘素材

工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

背景工单：你是某公司 AI 组的新运维。同事说他昨天在服务器上装了一个推理服务，今天早上启动失败，让你去处理。**机器上有个服务脚本 labservice.sh（见附录 A），它在某个目录下写日志。**

场景搭建手册（做题前执行）：

```
$ mkdir -p ~/lab && cd ~/lab
# 只检查并停止本场景记录的 PID：不要按模糊命令名误杀其他服务
$ if [ -f ~/lab/labservice.pid ]; then old_pid=$(sed -n '1p' ~/lab/labservice.pid); ps -p
"$old_pid" -o pid=,args=; fi
$ ./labservice.sh stop 2>/dev/null || true
$ nano labservice.sh      # 把附录 A1 的脚本内容全部粘贴进来，保存退出（Ctrl+X → Y → 回车）——
nano 是终端里的文本编辑器，比 vim 友好，新手先用它
$ nano labservice.conf    # 把附录 A2 的配置内容粘贴进来，LOGDIR 填你的真实路径（用 A2 里的生
成命令产生）
$ chmod +x labservice.sh
$ ./labservice.sh start &        # 后台启动（末尾加 &，不占终端），先确认它本来是好的
$ sleep 1 && ./labservice.sh status # 应显示：运行中：父进程 PID=…，端口=8080
$ ./labservice.sh stop           # 停掉它：子服务已停、PID 文件已清理（正常停止，日志不会写
ERROR）
$ ss -tlnp | grep 8080          # 无输出 = 端口已释放（确认干净后再继续）
# 现在注入故障：把 labservice.conf 里 PORT=8080 改成 PORT=abc（非法配置）
$ ./labservice.sh start          # 启动失败（返回非 0）——故障现场就位，开始做题
```

故障原理：日志目录里会留下 1 行真正的根因（[ERROR] 端口配置非法），其余都是干扰行。

编号	目标（只给这些）
进1-1	确认服务当前状态：进程在不在、端口有没有在监听
进1-2	找到日志文件的位置，从中定位出"根本原因那一行"，并说明你是怎么排除其他干扰行的
进1-3	修复问题，重新启动服务，让服务成功跑起来
进1-4	验证服务可用（端口通了、日志显示正常），写一份 3 行的处理记录（做了什么、为什么）

最终产出：一份故障处理记录 + 复述"处理流程四步"（即 《日志报错解读手册》的"读日志四步法"：确定时间窗口 → 收集日志/状态/退出码 → 结合上下文定位 → 修复并复测）。

场景 2｜磁盘满应急（模型文件占满磁盘）

背景工单：你负责的 GPU 服务器磁盘使用率报警到 92%，同事说服务开始写日志报"空间不足"。**你的任务：定位是什么占满了磁盘、安全清理、恢复服务、给出预防措施。**

搭建手册（做题前执行，需电脑 Ubuntu）：

```
# ① 先确认磁盘空间（df -h ~），再创建一个 500MB 空白镜像。
⚠ 空间紧张的替代参数（二选一，直接照抄整组；均为 Ubuntu 26.04 / ext4 / mount -o loop 实测
值）：
#   替代1：count=250（250MB 镜像）+ 塞入量合计 180MB，六个塞入行示例：30M、30M、30M、30M、
30M、30M。实测 df 约 219M/181M/21M、Use% 90%，以你机器 df 实测为准。
#   替代2：count=150（150MB 镜像）+ 塞入量合计 100MB，六个塞入行示例：20M、20M、20M、10M、
10M、20M。实测 df 约 125M/101M/14M、Use% 88%，以你机器 df 实测为准。
$ df -h ~
$ dd if=/dev/zero of=~/lab/disk.img bs=1M count=500
# ② 格式化成 ext4 文件系统（在镜像里格式化，真磁盘不受影响）
$ mkfs.ext4 ~/lab/disk.img
# ③ 挂载成独立分区（需要 sudo；loop 让文件伪装成磁盘）
$ sudo mkdir -p /mnt/lab
$ sudo mount -o loop ~/lab/disk.img /mnt/lab
# ④ 塞入"故障文件"：模型文件、旧日志、缓存各一堆，直到接近写满
$ sudo mkdir -p /mnt/lab/models /mnt/lab/logs /mnt/lab/cache
```

```
$ sudo fallocate -l 70M /mnt/lab/models/model_v1.bin
$ sudo fallocate -l 70M /mnt/lab/models/model_v2.bin
$ sudo fallocate -l 60M /mnt/lab/logs/old_service.log
$ sudo fallocate -l 60M /mnt/lab/cache/tmp_cache.bin
$ sudo fallocate -l 70M /mnt/lab/logs/old_service2.log
$ sudo fallocate -l 70M /mnt/lab/cache/tmp2.bin # 合计 400M；500M 镜像实测（Ubuntu）格式化后约 Size 452M、写满后 Used 约 401M、Avail 约 17M、Use% 约 96%。预期 Use% 约 95%~97%，以你机器的 df 实测为准——最后一条 fallocate 成功即搭建成功（Avail 剩十几 M 属正常：这正好复现“磁盘即将写满”）
$ df -h /mnt/lab        # 确认接近写满：预期 Use% 约 95%~97%，以你机器的 df 实测为准（>85% 即搭建成功），现场搭建完成
```

⚠ 安全提示：故障全部发生在 /mnt/lab 这个镜像分区里。做完题执行 `sudo umount /mnt/lab` 卸载后，一切恢复原样，真系统没有任何变化。

题目：进2-1 确认是哪个挂载点满了，占用多少。进2-2 进入该分区，逐层定位最大的目录、最大的文件。进2-3 判断哪些可安全删除、哪些不能动（说清依据），完成清理。进2-4 验证恢复，写出 2 条预防措施。

场景 3｜服务连不上（网络链路排查）

本关三要素（编号 ｜ 时长 ｜ 完成判定）

1-40 • 场景 3（上）：复现"服务连不上" ★ ｜ 休息日 50 分 ｜ 工作日 15 分

□ 复现故障，列出你要先看的三件事

回滚方式：同上

1-41 • 场景 3（中）：四层链路定位 ★ ｜ 休息日 50 分 ｜ 工作日 15 分

□ 按主机→端口→进程→日志定位到根因

1-42 • 场景 3（下）：修复 + 记录 ★ ｜ 休息日 50 分 ｜ 工作日 15 分

□ 恢复服务并写完记录

回滚方式：恢复现场

工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：同事反馈"我们部署在本地 8080 端口的服务突然连不上了"，让你按链路排查：**主机通不通 → 端口通不通 → 服务进程在不在 → 日志怎么说。**

🛠 **场景搭建手册**：在本机用 labservice.sh 起一个服务，然后从三种故障里**任选一种注入**（做两遍可以换着来）：

```
$ cd ~/lab
# 故障 A：服务压根没起（什么都不做，直接进入题目）
# 故障 B：服务起了，但端口被别的进程占了：
$ python3 -m http.server 8080 &      # 先用 Python 起一个简易网页服务，占住 8080 端口
HTTP_PID=$!                          # 记下它的 PID，做完题只停它，别误伤别的 Python 服务
$ ./labservice.sh start               # 服务启动失败 → 端口冲突
# 故障 C：服务正常但监听了错误的地址——把 labservice.sh 里这一行：
#   python3 -m http.server "$PORT"
# 改为（--bind = 指定只监听哪个地址；此写法让它只监听本机回环地址）：
#   python3 -m http.server --bind 127.0.0.1 "$PORT"
# 然后用"另一台设备"访问这台机器的局域网 IP 测试（单机用户可用平板连电脑 IP；
# 不方便时跳过 C，只做 A、B 两种故障）单机自测法：没有第二台设备时，用 ip addr 查本机局域网
IP，再 nc -zv 本机IP 8080——同样得到 refused，不用真找另一台设备。
```

新命令介绍：**nc**（netcat，网络工具）——nc -zv 主机 端口 用来测试"端口到底通不通"。若提示 nc 不存在，先 **sudo apt install netcat-openbsd**。

编号	目标（只给这些）
进3-1	用 **ping**（测试主机可达性）确认主机可达性（含 ping localhost 与 ping 网关的区别）
进3-2	用 **ss -tlnp**（查看监听端口）查看监听端口，判断目标端口有没有在听、被谁听
进3-3	用 **nc -zv** 或 **curl** 实测端口通不通，把"进程在听"和"真的能连上"分开判断
进3-4	定位故障点，修复，并复述分层排查：回环/网关 → 目标主机 → 端口/应用 → 进程/日志，同时说明每层结论的边界

最终产出：一份网络故障排查记录 + 画一张排查顺序流程图。

场景 4｜服务与进程运维（含 GPU 进程定位）

本关三要素（编号 ｜ 时长 ｜ 完成判定）

1-43 • 场景 4（上）：进程与资源占用排查 ★ ｜ 休息日 50 分 ｜ 工作日 15 分

□ 找出占资源最多的进程并说清依据

回滚方式：只对练习进程下手

1-44 • 场景 4（中）：定位与安全处理 ★ ｜ 休息日 50 分 ｜ 工作日 15 分

□ 定位根因并安全处理

回滚方式：恢复现场

工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：你在管一台"推理服务机器"，今天要完成：部署一个服务并让它开机自启、监控它、处理一个"卡死进程"，最后查清楚"显卡/文件被哪个进程占着"。

📌 环境边界（实测结论）：① **GPU 题（进4-3）是 NVIDIA 外部扩展**：需 NVIDIA 显卡 + 驱动 + nvidia-smi；fuser 一般已自带，若提示找不到再装 sudo apt install psmisc。**无 GPU 就做替代题**（fuser -v 查文件占用），**替代题通过 ≠ GPU 题通过**。② **systemd 题（进4-4 开机自启）**：完整验证需 Ubuntu 主机 systemd（**Restart=on-failure 已在 Ubuntu 26.04 + systemd 实测通过：外部 kill -9 子服务后 unit 自动重启（MainPID 变化、NRestarts 递增、端口恢复）；systemctl stop 后 unit inactive、不误触发重启**）；WSL 可能可用，容器 PID 1 非 systemd 时 systemctl 为 offline，不能替代主机验收。实验边界：Ubuntu + systemd + sudo + 专用实验机，完整走 start → 验证 → stop → 清理；正常 stop 后 unit 应为 inactive。受限环境都如实记为“环境限制”，不影响其余题目。

场景搭建手册：

```
$ cd ~/lab
$ nano loop.sh                          # 粘贴附录 A3 的完整内容（死循环进程），保存退出（Ctrl+X → Y → 回车）
$ chmod +x loop.sh                      # 给执行权限；⚠ loop.sh 跑起来会占满一个 CPU 核，用完必须 kill 并收尾
$ ./labservice.sh start &                  # 只启动本场景服务
$ nohup ./loop.sh > loop.log 2>&1 & echo $! > loop.pid # 保存 loop.sh 的 PID
$ top                                       # 现场就位
```

收尾：读取 loop.pid，先用 ps -p PID -o pid=,args= 核对命令归属，再按 PID 停止；执行 ./labservice.sh stop 停止本场景服务；最后删除 loop.pid、loop.log 和服务 PID 文件。本练习不使用按模糊命令名清理的方式。

编号	目标（只给这些）
进4-1	部署服务：后台运行 + 确认进程和端口都正常
进4-2	用 top/ps 找出那个死循环进程，优雅终止它（并说明为什么不直接 -9）
进4-3	GPU 题：用 `nvidia-smi` 查看显卡状态，用 `fuser -v` 定位占用显卡的进程并处理。/ 无 GPU 替代题：定位"占用某个文件"的进程并处理（技能完全等价）
进4-4	设置服务开机自启（systemd 写 unit 文件 / 或 rc.local / 或说明其他方式），重启验证自启生效

最终产出：一台"服务能自启、能监控、出问题会处理"的练习机状态 + 处理记录。

场景 5 | 脚本收尾（衔接 Shell 脚本任务）

本关三要素（编号 | 时长 | 完成判定）

2-01 • 五件套地图：先看懂 5 个脚本各解决什么问题 ★ | 休息日 1 小时 | 工作日 15 分

□ 能用一句话说出每个脚本解决什么问题

2-02 • backup.sh：备份 + 日期后缀 ★ | 休息日 2 小时 | 工作日 15 分

□ 脚本能生成带日期的备份文件

回滚方式：只写 ~/lab；先拿测试目录试跑一遍（没有 --dry-run 开关——直接加会被当成路径名报错）

2-03 • backup.sh 排错：路径空格与变量引用 ★ | 休息日 1.5 小时 | 工作日 15 分

□ 故意造一个带空格的文件名，脚本仍能处理

2-04 • disk_alert.sh：取数 + 阈值判断 ★ | 休息日 2 小时 | 工作日 15 分

□ 磁盘超 80% 时输出告警

实战价值：磁盘告警脚本怎么写是常见追问

2-06 • service_guard.sh：服务挂了自动拉起 ★ | 休息日 2 小时 | 工作日 15 分

□ 手动停服务后，脚本能把它拉起来

回滚方式：只对自己起的练习服务做

2-07 • 把脚本变成 systemd 服务 ★ | 休息日 2 小时 | 工作日 15 分

□ systemctl status 能看到自己的服务

回滚方式：不用时 systemctl disable 掉

工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

背景工单：前四个场景你都是手动敲命令处理的。现在公司要求把巡检"脚本化"：以后一键就能看到机器状态。**这就是通往"5 个 Shell 脚本"任务的桥。**

编号	目标（只给这些）
进5-1	写一个"资源巡检"脚本：一键输出磁盘占用（df）、内存（free）、CPU 负载（uptime）摘要
进5-2	写一个"服务健康检查"脚本：检查进程在不在 + 端口通不通 + 日志最近有没有更新，结果用中文提示输出

编号	目标（只给这些）
进5-3	给两个脚本加上变量、判断和注释，让一个完全没写过脚本的人能看懂
进5-4	运行验证两个脚本的输出正确，并（可选）用 `cron`（定时任务）设置每天自动巡检一次

最终产出：2 个可用的 Shell 脚本，直接衔接"5 个 Shell 脚本"任务——5 件套总览（本场景产出前 2 个，后 3 个按表补齐）：

脚本	功能	知识来源	优先级
`check_res.sh`	资源巡检（磁盘/内存/负载 + 日期）	本场景进5-1	★ 必做
`health_check.sh`	服务健康检查（进程/端口/日志时间）	本场景进5-2	★ 必做
`backup.sh`	自动备份 + 日期后缀（tar + date）	基础篇 5-6 + 教科书第 12 章	★ 必做
`disk_alert.sh`	磁盘超 80% 告警（df 解析 + if 判断）	进5-1 思路 + 教科书第 13 章	★ 必做
`service_guard.sh`	服务挂了自动拉起（systemctl is-active 检查 || start）	教科书第 6、10 章	★ 必做

另有"批量建用户""清理过期日志"两项能力已在基础篇阶段 3、阶段 5 覆盖，不再单独成脚本。全部脚本配 README，保存到 shell-scripts 仓库。

（本叠是题目，参考答案在另一叠；卡住 20 分钟以上再翻）